TECHNICAL ASSESSMENT REPORT

Full Stack Developer Assessment — ZappySales Project

Candidate Name:	Ngane Emmanuel
Role:	Full Stack Developer Assessment
Project Name:	ZappySales User & Address Management
Date of Submission:	June 17, 2026

---

1. Overview

ZappySales is a full-stack, production-quality User and Address Management application built with Spring Boot and React. It acts as a lightweight administrative directory that allows system administrators to view, search, paginate, and modify user profiles and their associated shipping/billing addresses.

The application follows the **Aggregate Root** pattern: the User is the parent record, and multiple Address items are managed directly within the User's context (a 1-to-many relationship).

This assessment implementation goes beyond a minimal proof-of-concept by introducing:

- **Server-side Pagination and Case-insensitive Substring Search** (supporting enterprise-scale data sets).
- **Environment-driven properties** with standard defaults for cloud-native deployment.
- **IP-based in-memory Rate Limiting / Request Throttling** on the backend.
- **35 Vitest + React Testing Library (RTL) tests** on the frontend.
- **47 JUnit 5 + Mockito + MockMvc unit/integration tests** on the backend.
- **OpenAPI/Swagger Interactive Console** for easy endpoint verification.

2. Technologies

Backend (Java 21 / Spring Boot)

- **Language/Framework:** Java 21, Spring Boot v4.1.0.
- **Thread-safe Runtime Store:** Managed in-memory via ConcurrentHashMap with defensive cloning during reads and writes to isolate repository state.
- **Data Transfer Objects (DTOs):** Strong contracts separate the external API model from internal domain models.
- **API Validation:** Jakarta Bean constraints integrated with custom HTML input sanitizers (@SanitizedString).
- **Security & Throttling:** Custom servlet filter applying security headers and IP-based rate limiting filter.

Frontend (React 19 / TypeScript)

- **Framework & Tooling:** React 19, Vite 8, TypeScript 6.
- **Styling System:** TailwindCSS v4.0 for utility layout grids and layout properties combined with Material-UI (MUI) v9 for UI component blocks.
- **State Management:** Zustand state store (userStore.ts) providing reactive global flow with minimal boilerplate.
- **Data Fetching:** Standardized Axios client wrapper with custom request/response interceptors.

3. Directory Layout

```
ZappySales/
■■■ backend/                        # Spring Boot Project
■   ■■■ src/main/java/              # Core Source Code
■   ■■■ src/main/resources/         # application.properties
■   ■■■ src/test/java/              # JUnit Test Suites
■■■ frontend/                       # React + TypeScript Client
■   ■■■ src/
■   ■   ■■■ api/                    # Centralized Axios Client
■   ■   ■■■ app/routes/             # URL Routing Layouts
■   ■   ■■■ features/users/         # Feature Pages, Components, Stores, and Hooks
■   ■   ■■■ shared/                 # Common Page Skeletons, Layouts, and Spinners
■   ■■■ test/                       # Vitest Setup Configs
■   ■■■ vitest.config.ts            # Vitest Config File
■■■ README.md                       # Setup and Running Guide
```

4. Features

- **User CRUD** - Full capability to view, register, edit, and delete user profiles.
- **Address CRUD** - Management of multiple addresses associated with each user (1-to-many relationship).
- **Server-side Pagination** - Seamless pagination on data grids to support enterprise-scale databases.
- **Search** - Case-insensitive substring search matching email, first name, or last name.
- **Input Sanitization** - Custom @SanitizedString annotation to prevent XSS.
- **Rate Limiting** - In-memory sliding-window IP-based API throttling filter.
- **Security Headers** - Custom response headers (X-Content-Type-Options, X-Frame-Options, Referrer-Policy).
- **Swagger API Docs** - Automated interactive developer UI console.
- **Unit and Integration Tests** - Comprehensive test coverage (47 backend, 35 frontend).

5. Architecture

Backend: `Controller → Service → Repository → In-memory Storage`

Frontend: `Pages → Hooks → Store → Services → Axios Client → REST API`

6. Environment & Prerequisites

Prerequisites: JDK 21, Node.js (v18+), Maven.

Environment Variable	Description	Default Value
`PORT`	The HTTP port the backend server listens on	`8080`
`ALLOWED_ORIGINS`	Comma-separated list of permitted CORS origins	`http://localhost:3000,http://localhost:5173`
`RATE_LIMIT_CAPACITY`	Max API requests allowed per client IP within the window	`100`
`RATE_LIMIT_TIME_WINDOW_SECONDS`	Throttling time window in seconds	`60`

Frontend Configuration: Create frontend/.env with: VITE_API_BASE_URL=http://localhost:8080/api/v1

7. Setup Instructions

Backend (in backend/ folder):

```
mvn clean install
./mvnw spring-boot:run
```

Frontend (in frontend/ folder):

```
npm install
npm run dev
npm run build
```

8. Testing Instructions

Backend: mvn test (in backend/ folder)

Frontend: npm run test (in frontend/ folder)

9. Swagger Documentation

```
Swagger UI console is available locally at: http://localhost:8080/swagger-ui/index.html
```

10. Design Choices: "User → Address" Flow

- **Aggregate Root UI:** Administrators open the main user directory list page and navigate to a dedicated detail page, keeping the detail management focused.
- **In-Context Modals:** CRUD tasks (edit profile, CRUD addresses) trigger popup dialogs in-context on the detail view rather than redirecting away, yielding a smooth single-dashboard flow.
- **Zustand Sync:** Successful changes trigger local store mutations rather than forcing database refetches, reducing network utilization.

11. Test Suite Validation Results

- **Backend (47 tests passed):** Checked repository deep cloning, controller endpoints, validation, and full user integration workflows.
- **Frontend (35 tests passed):** Checked table renders, dialog state triggers, Zustand stores, and routing page displays.

Candidate Signature: Ngane Emmanuel • Full Stack Developer